

```
import kagglehub
atharvaingle_crop_recommendation_dataset_path = kagglehub.dataset_download('atharvaingle/crop-recommendation-dataset')

print('Data source import complete.')
```

Using Colab cache for faster access to the 'crop-recommendation-dataset' dataset.
Data source import complete.

```
import plotly.offline as pyo

import matplotlib.pyplot as plt
import seaborn as sns

from wordcloud import WordCloud, STOPWORDS
from PIL import Image

import plotly.graph_objs as go
import pandas as pd
import numpy as np

import warnings
warnings.filterwarnings("ignore")

pd.options.display.max_columns = None

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

/kaggle/input/crop-recommendation-dataset/Crop_recommendation.csv

First look at the dataset We have 2200 rows and 8 columns in the dataset with no missing values.

All values in the data are numerical except for the label data.

```
df = pd.read_csv('/kaggle/input/crop-recommendation-dataset/Crop_recommendation.csv')
df.head()
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

```
df.shape
```

```
(2200, 8)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0    N                2200 non-null   int64
1    P                2200 non-null   int64
2    K                2200 non-null   int64
3    temperature      2200 non-null   float64
4    humidity         2200 non-null   float64
5    ph               2200 non-null   float64
6    rainfall         2200 non-null   float64
7    label           2200 non-null   object
dtypes: float64(4), int64(3), object(1)
memory usage: 137.6+ KB
```

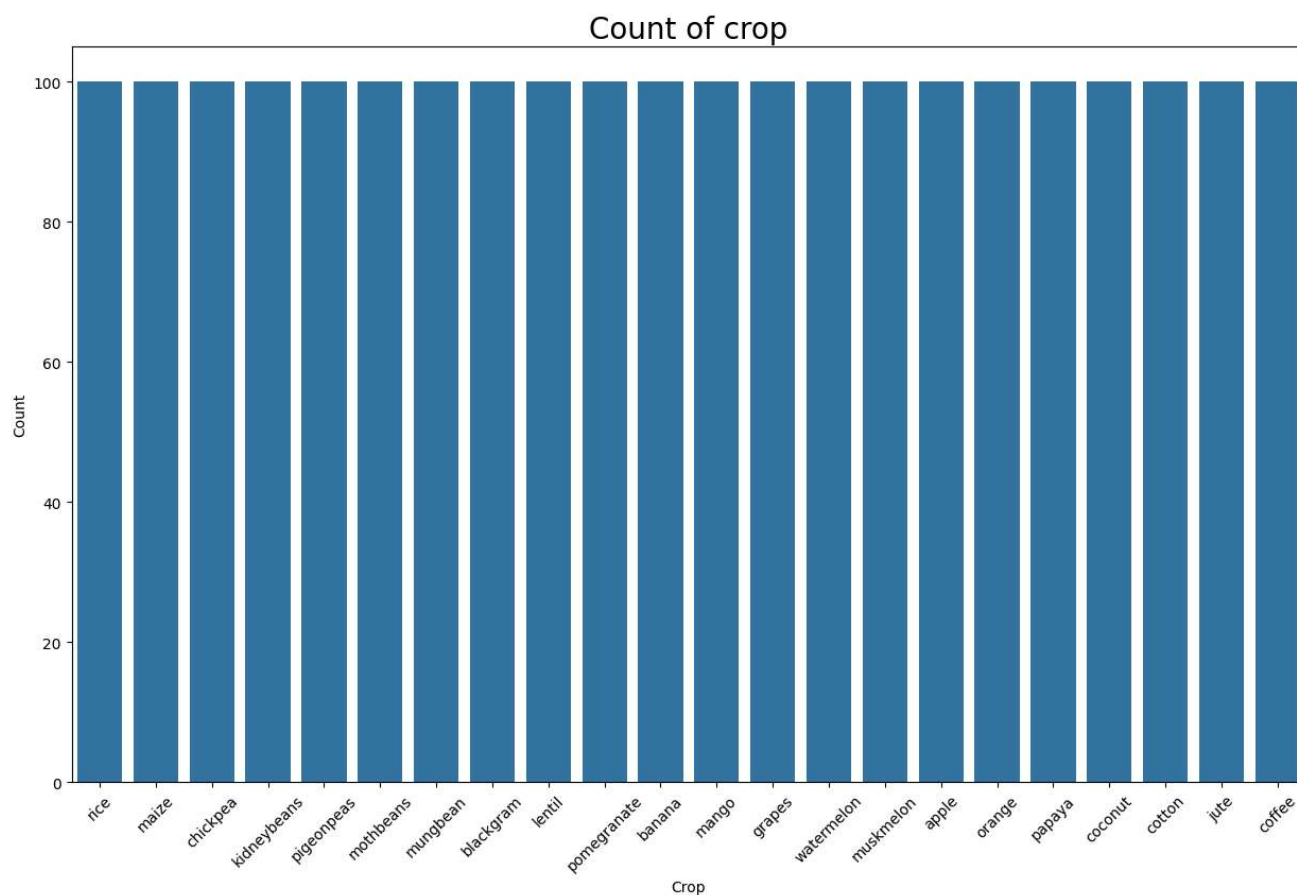
```
df.describe()
```

	N	P	K	temperature	humidity	ph	rainfall
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000
mean	50.551818	53.362727	48.149091	25.616244	71.481779	6.469480	103.463655
std	36.917334	32.985883	50.647931	5.063749	22.263812	0.773938	54.958389
min	0.000000	5.000000	5.000000	8.825675	14.258040	3.504752	20.211267
25%	21.000000	28.000000	20.000000	22.769375	60.261953	5.971693	64.551686
50%	37.000000	51.000000	32.000000	25.598693	80.473146	6.425045	94.867624
75%	84.250000	68.000000	49.000000	28.561654	89.948771	6.923643	124.267508
max	140.000000	145.000000	205.000000	43.675493	99.981876	9.935091	298.560117

A very balanced dataset

```
fig, ax = plt.subplots(1, 1, figsize=(15, 9))
sns.countplot(data=df, x='label')
ax.set(xlabel='Crop')
ax.set(ylabel='Count')
plt.xticks(rotation=45)

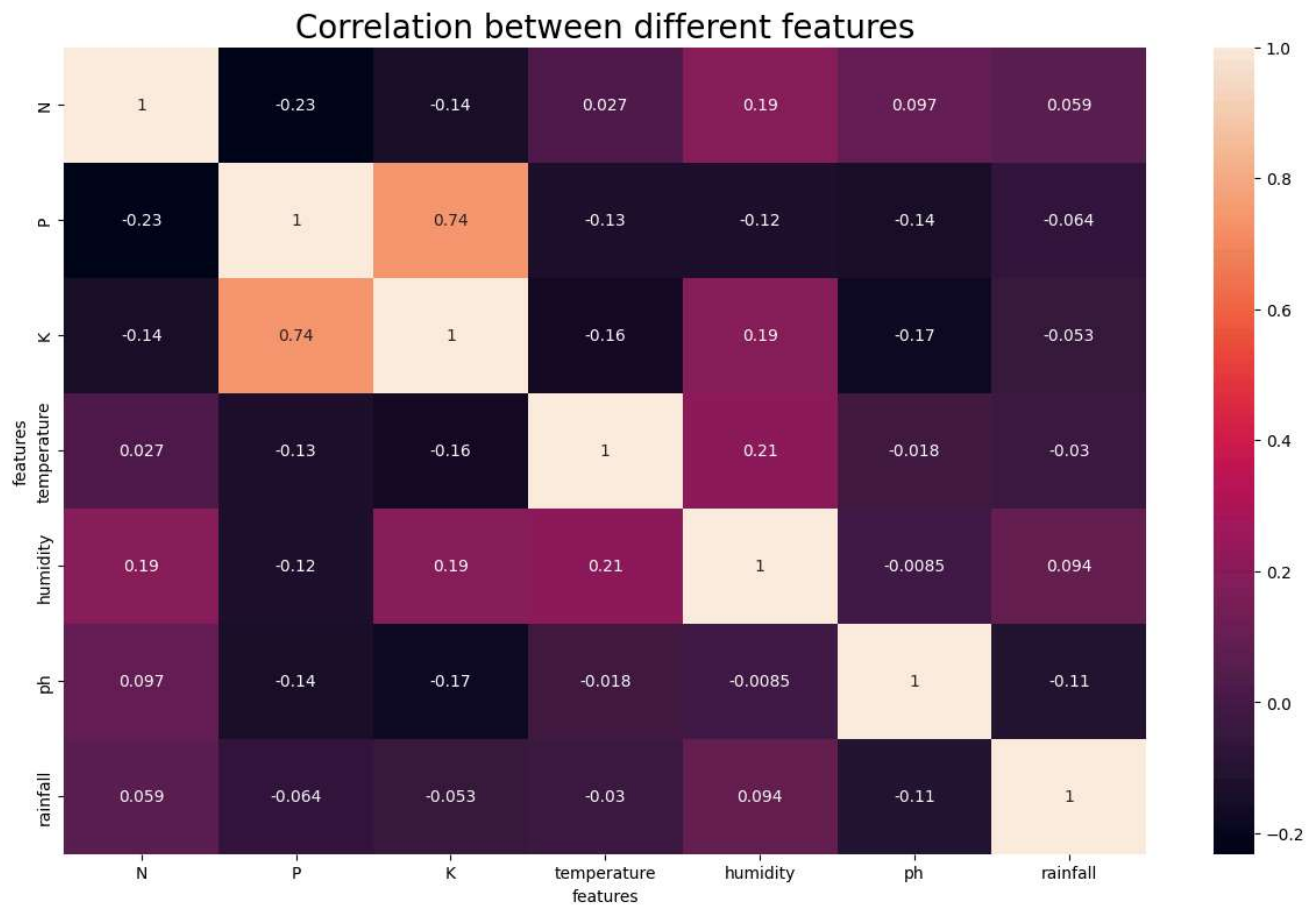
plt.title('Count of crop', fontsize = 20, c='black')
plt.show()
```



Correlation between different figures Apart from between P and K, there is low correlation between the features.

```
fig, ax = plt.subplots(1, 1, figsize=(15, 9))
sns.heatmap(df.drop('label', axis=1).corr(), annot=True)
ax.set(xlabel='features')
ax.set(ylabel='features')

plt.title('Correlation between different features', fontsize = 20, c='black')
plt.show()
```



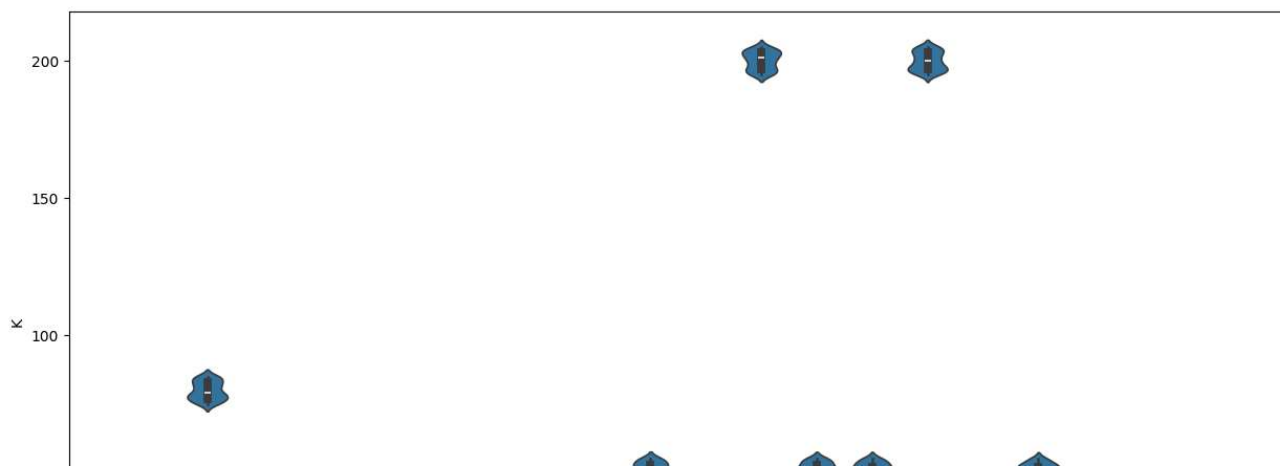
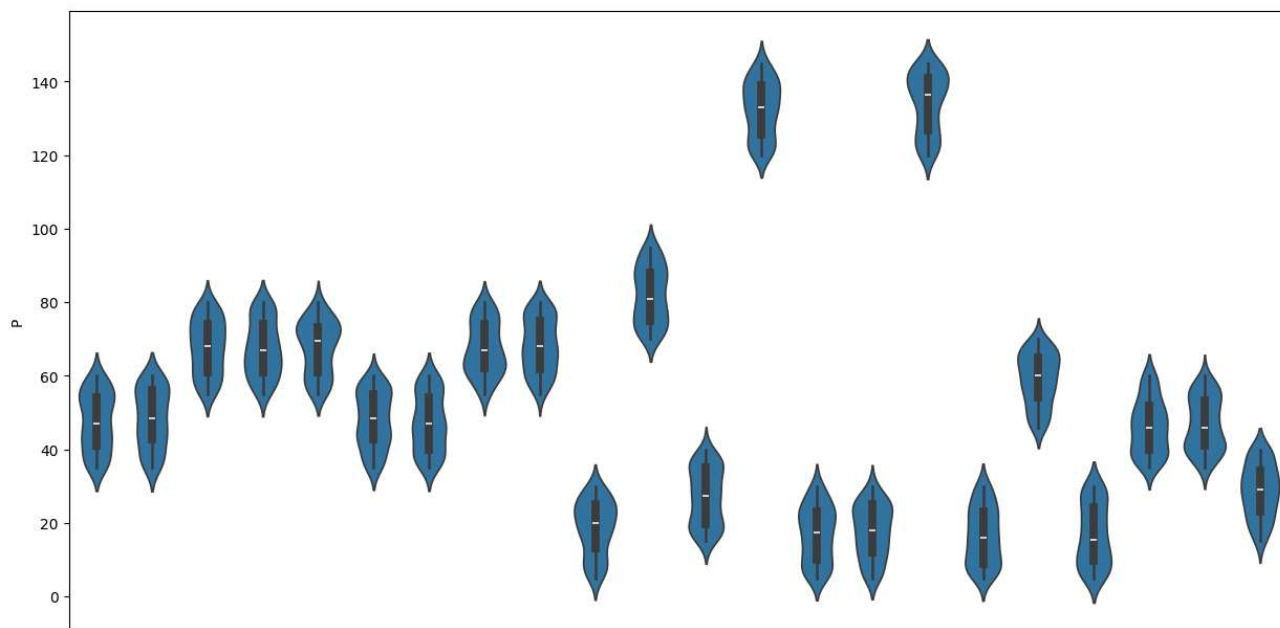
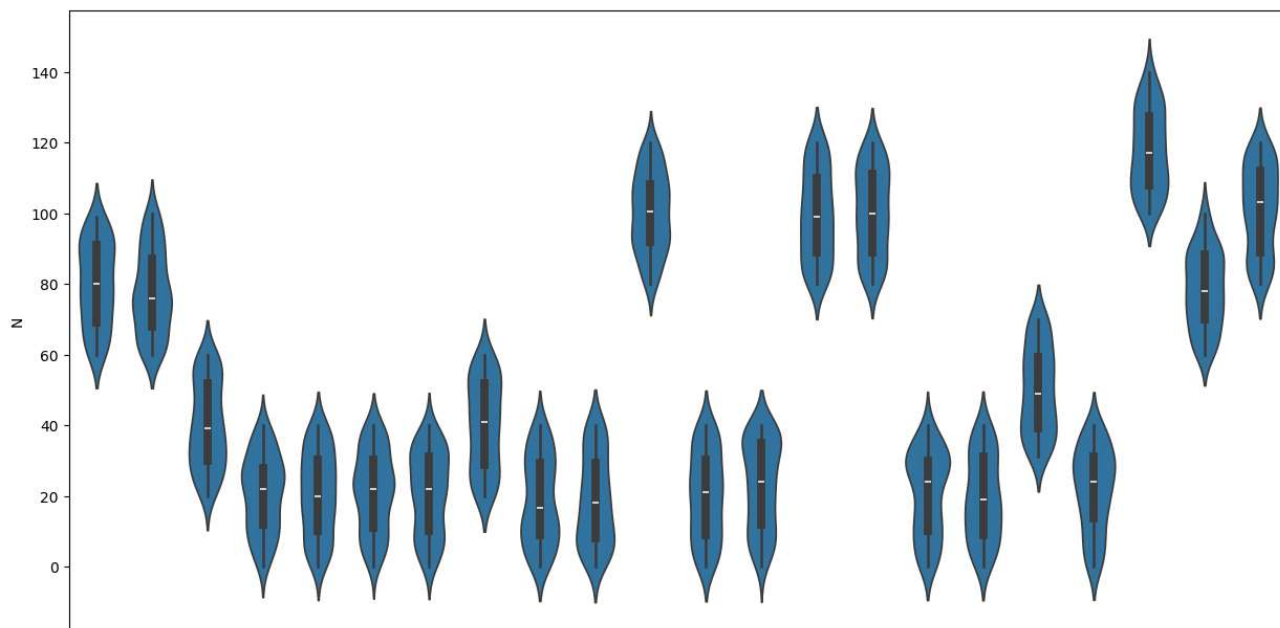
Violin plots of all features!

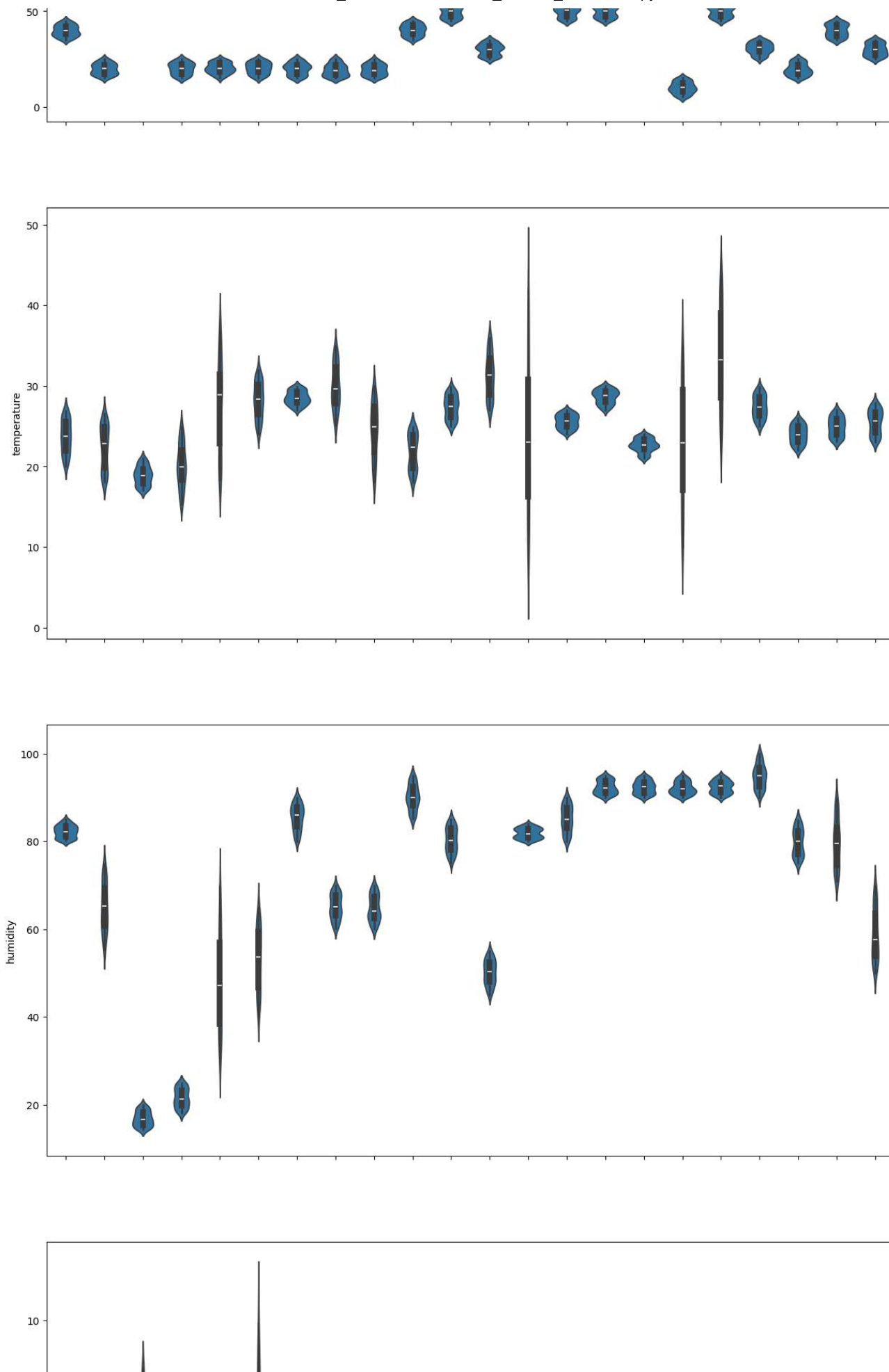
```
columns = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']

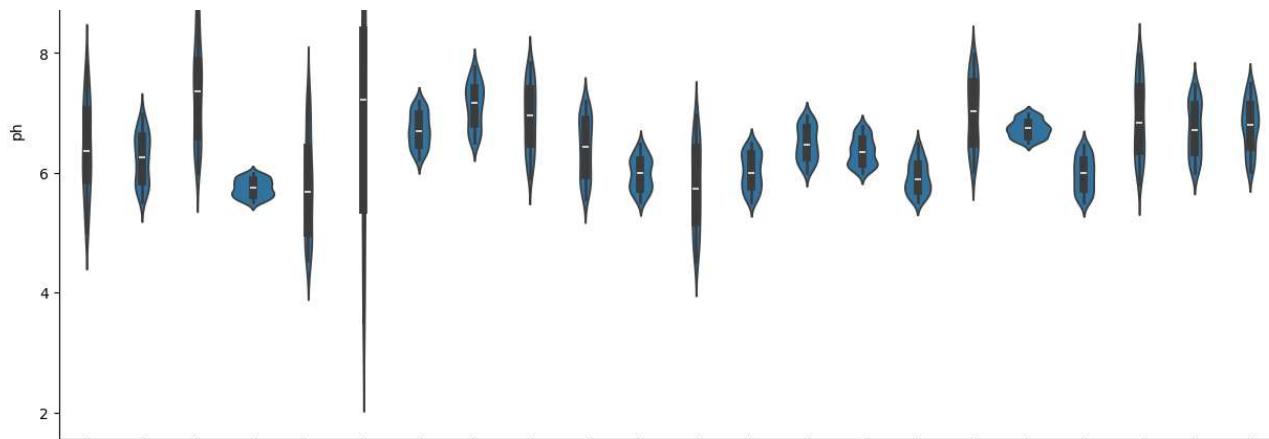
fig, ax = plt.subplots(7, 1, figsize=(15, 63), sharex=True)

i = 0
for column in columns:
    sns.violinplot(data=df, x='label', y=column, ax=ax[i])
    plt.xticks(rotation=45)
    i = i + 1

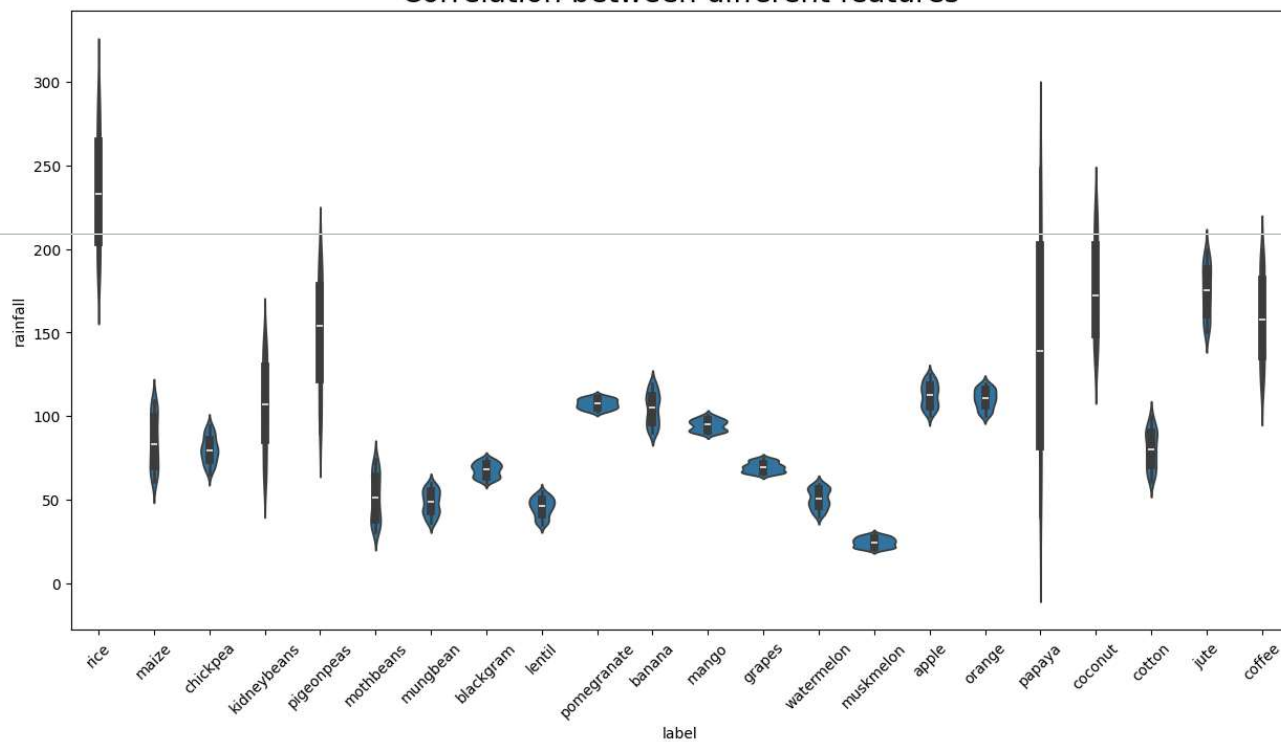
plt.title('Correlation between different features', fontsize = 20, c='black')
plt.show()
```





Correlation between different features



Building the model

```

from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from xgboost.sklearn import XGBClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.preprocessing import LabelEncoder # Add this import

target = ['label']
features = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']

X = df[features]

# Encode categorical labels to numerical labels
le = LabelEncoder()
y = le.fit_transform(df[target].values.ravel()) # .ravel() converts to 1D array as expected by LabelEncoder

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42, shuffle=True)

models = []
models.append(('LogisticRegression', LogisticRegression(random_state=0)))
models.append(('DecisionTreeClassifier', DecisionTreeClassifier(random_state=0)))
models.append(('XGBClassifier', XGBClassifier(random_state=0)))
models.append(('GradientBoostingClassifier', GradientBoostingClassifier(random_state=0)))
models.append(('KNeighborsClassifier', KNeighborsClassifier()))
models.append(('RandomForestClassifier', RandomForestClassifier(random_state=0)))

model_name = []
accuracy = []

for name, model in models:
    model.fit(X_train,y_train)
    y_pred=model.predict(X_test)
    model_name.append(name)
    accuracy.append(metrics.accuracy_score(y_test,y_pred))
    print(name, metrics.accuracy_score(y_test,y_pred))

```

```

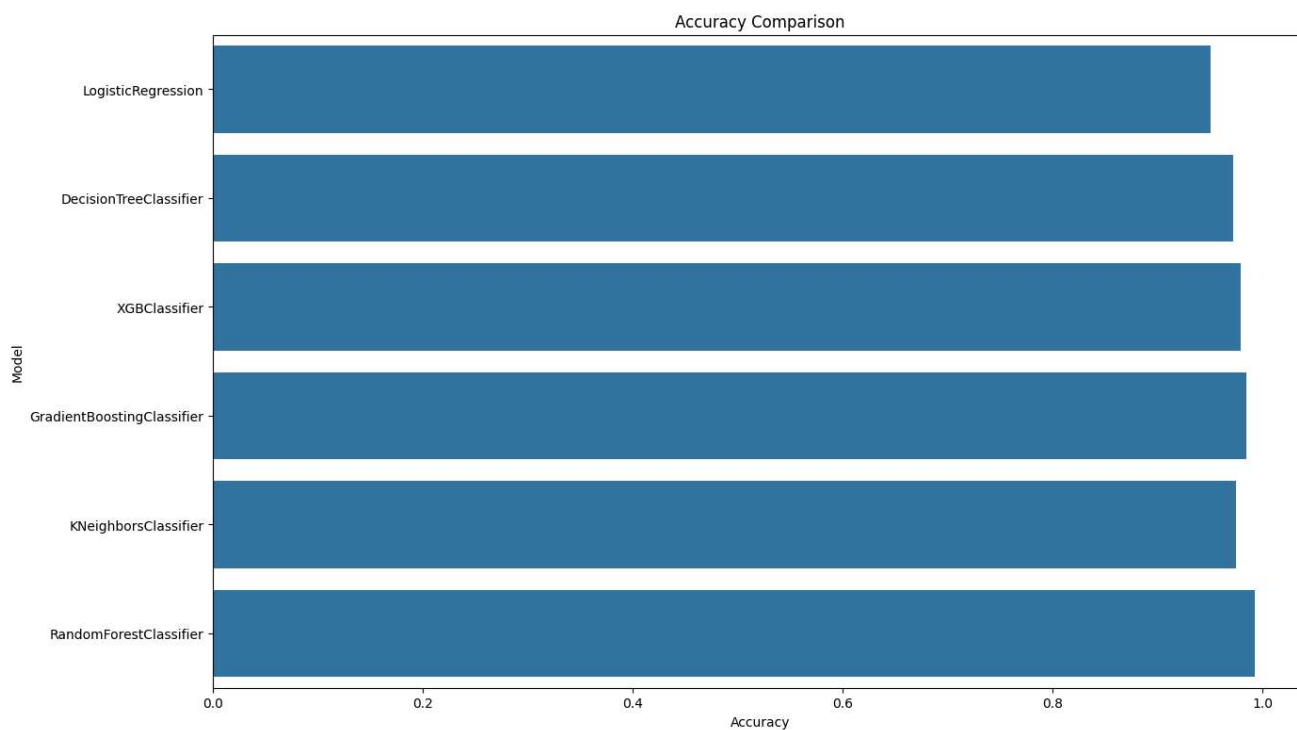
LogisticRegression 0.9504132231404959
DecisionTreeClassifier 0.9724517906336089
XGBClassifier 0.9793388429752066
GradientBoostingClassifier 0.9848484848484849
KNeighborsClassifier 0.9752066115702479
RandomForestClassifier 0.9931129476584022

```

```

plt.figure(figsize=(15,9))
plt.title('Accuracy Comparison')
plt.xlabel('Accuracy')
plt.ylabel('Model')
sns.barplot(x = accuracy, y = model_name)
plt.show()

```

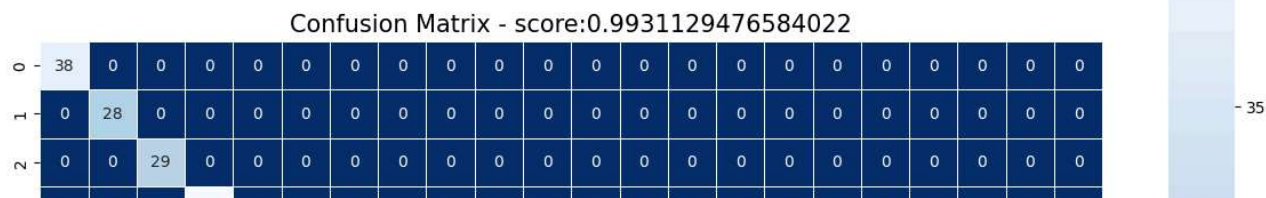
```

model=RandomForestClassifier(random_state=0)
model.fit(X_train,y_train)
y_pred=model.predict(X_test)

from sklearn import metrics

cm = metrics.confusion_matrix(y_test, y_pred)
plt.figure(figsize=(15,15))
sns.heatmap(cm, annot=True, fmt=".0f", linewidths=.5, square = True, cmap = 'Blues_r');
plt.ylabel('Actual label');
plt.xlabel('Predicted label');
all_sample_title = 'Confusion Matrix - score:'+str(metrics.accuracy_score(y_test,y_pred))
plt.title(all_sample_title, size = 15);
plt.show()
print(metrics.classification_report(y_test,y_pred))

```

```
# =====
# CROP RECOMMENDATION SYSTEM - DETAILED & IMPROVED VERSION
# =====

# STEP 1: Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)
from sklearn.inspection import permutation_importance

import joblib
import warnings
warnings.filterwarnings('ignore')

print("✅ Libraries imported successfully!")

# =====
# STEP 2: Upload & Load Dataset
# =====
from google.colab import files

print("\n📁 Please upload 'crop_recommendation.csv'")
uploaded = files.upload()

data = pd.read_csv("/content/Crop_recommendation.csv")

# =====
# STEP 3: Detailed Data Exploration (EDA)
# =====
print("\n" + "="*60)
print("📊 EXPLORATORY DATA ANALYSIS (EDA)")
print("="*60)

print(f"\n ♦ Dataset Shape: {data.shape[0]} rows × {data.shape[1]} columns")

print("\n ♦ First 5 Rows:")
print(data.head())

print("\n ♦ Data Types:")
print(data.dtypes)

print("\n ♦ Statistical Summary:")
print(data.describe().round(2))

print("\n ♦ Unique Crops in Dataset:")
print(data['label'].unique())
print(f"    Total unique crops: {data['label'].nunique()}")

print("\n ♦ Crop Distribution (samples per crop):")
print(data['label'].value_counts())

# =====
# STEP 4: Data Quality Checks
# =====
print("\n" + "="*60)
print("🔍 DATA QUALITY CHECKS")
print("="*60)
```